

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x06 实验报告

硬盘驱动与文件系统

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. MBR 分区表

首先我们需要根据 MBR 的结构定义，利用 `MbrPartition` 的 `parse` 方法来对四个 `partition` 依次解析，如下

```
for i in 0..4 {
  let offset = 0x1BE + i * 16;
  partitions.push(
      // FIXED: parse the mbr partition from the buffer
      //      - just ignore other fields for mbr
      MbrPartition::parse(buffer[offset..offset + 16].try_into().unwrap()),
  );
```

然后我们需要利用 `define_filed!` 完成内部数据的解析提取，由于 `CHS` 的 `start` 和 `end` 中的 `sector` 和 `cylinder` 不是占据整个字节的，因此要另外写 `start_sector` 等函数进行解析提取，部分代码如下

```
define_field!(u8, 0x00, status); // Status (bit 7 set = active or bootable)
define_field!([u8; 3], 0x01, chs_start); // CHS address of first partition sector
(3 bytes)
define_field!(u8, 0x04, partition_type); // Partition type
define_field!([u8; 3], 0x05, chs_end); // CHS address of last partition sector (3
bytes)
define_field!(u32, 0x08, begin_lba); // LBA of partition start
define_field!(u32, 0x0C, total_lba); // Number of sectors in partition

pub fn is_active(&self) -> bool {
    self.status() == 0x80
}

pub fn begin_head(&self) -> u8 {
    self.chs_start()[0]
}

pub fn begin_sector(&self) -> u8 {
    self.chs_start()[1] & 0x3F
}

pub fn begin_cylinder(&self) -> u16 {
    ((self.chs_start()[1] as u16 & 0xC0) << 2) | self.chs_start()[2] as u16
}

// ..... end_head(), end_sector()... is the same as above,
// see entry.rs for the full code
```

然后我们对这部分进行功能测试，需要在根目录的 `Cargo.toml` 的 `member` 中加入 `"crates/storage",` 表项，然后再 `storage/` 下运行 `cargo test`，可以看到，可以通过对应的测试，说明实现大致无误。

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage$ cargo test -- --nocapture
running 1 test
Partition Meta Data {
    Active: true,
    Begin Head: "0x01",
    Begin Sector: "0x0001",
    Begin Cylinder: "0x0000",
    Partition Type: "0x0b",
    End Head: "0xfe",
    End Sector: "0x003f",
    End Cylinder: "0x02fc",
    Begin LBA: "0x0000003f",
    Total LBA: "0x00bb867e",
}
test partition::mbr::entry::tests::partition_test ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00
s

   Doc-tests ysos_storage

running 1 test
test crates/storage/src/common/macros.rs - common::macros::define_field (line 5) ... ignored

test result: ok. 0 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out; finished in 0.00
s

all doctests ran in 0.40s; merged doctests compilation took 0.40s
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage$
```

2. ATA 磁盘驱动

我们要实现的是 LBA（Logical Block Address, 逻辑块寻址） 读写命令和 `IdentifyDevice` 这三个命令，为了方便实现，封装了一个发送命令并等待设备就绪的函数 `write_command()`。

2.1. 发送命令

对于 `write_command()`，我们先根据 `block` 的值来解析出 `lba` 地址的低中高八位，然后设置 `drive` 寄存器的值。其中 `drive` 寄存器低四位放在 `block` 高四位，而 `drive` 的高三位需要设为 `1`（第六位表示选择 LBA 寻址方式），第五位为主/从设备的 `drive` 选择位。同时把命令写入 `command` 寄存器。

然后等待设备就绪，判断是否出错，随后等待数据请求就绪（虽然我们这里只实现了读写和 `IdentifyDevice` 三个命令，但由于 `AtaCommand` 里面还有很多别的命令是不需要等待数据请求就绪的，因此这里加了个特判） `FIXME` 部分代码如下

```
// FIXED: store the LBA28 address into four 8-bit registers
//      - read the documentation for more information
//      - enable LBA28 mode by setting the drive register
self.lba_low.write(bytes[0]);
self.lba_mid.write(bytes[1]);
self.lba_high.write(bytes[2]);

self.drive.write(0xE0 | (drive << 4) | (bytes[3] & 0x0F));
// FIXED: write the command register (cmd as u8)
self.command.write(cmd as u8);

// FIXED: poll for the status to be not BUSY
self.poll(AtaStatus::BUSY, false);

// FIXED: poll for the status to be not BUSY and DATA_REQUEST_READY
if cmd == AtaCommand::ReadPio
    || cmd == AtaCommand::WritePio
    || cmd == AtaCommand::IdentifyDevice
{
    self.poll(AtaStatus::BUSY, false);
    self.poll(AtaStatus::DATA_REQUEST_READY, true);
}
```

2.2. 磁盘识别

首先我们要实现的是 `identify_drive()` 函数，根据 `FIXME` 中的提示补全即可，具体请见 `ata/bus.rs`。

然后，我们要实现的是 `open` 函数，来获取磁盘信息，我们要获取的部分是 `serial`，`model` 以及 `blocks` 的值，首先前面两个我们可以通过取 `buf` 中的对应段即可（因为 `X86` 架构是小端序，需要用 `to_be_bytes` 来获取正确的端序）。

但是如果我们直接在 `buf` 中对应的段取出 `blocks` 的值，会得到 `120.00009 GiB` 的输出，这是因为解析数值的时候也会按照小端序来解析，我们可以直接根据原始数据 `res` 中的对应两个 `word` 来拼出 `blocks` 正确的值。代码如下

```rust
let serial = {
    /* FIXED: get the serial from buf */
core::str::from_utf8(&buf[ATA_IDENT_SERIAL..ATA_IDENT_SERIAL + 20])
        .unwrap_or("")
        .trim()
        .into()
};
let model = {
    /* FIXED: get the model from buf */
core::str::from_utf8(&buf[ATA_IDENT_MODEL..ATA_IDENT_MODEL + 40])
        .unwrap_or("")
        .trim()
        .into()
};
let blocks = {
    /* FIXED: get the block count from buf */
    ((res[61] as u32) << 16) | (res[60] as u32)
};
```

然后我们再 `lib.rs` 中加入 `AtaDrive::open(0, 0)`，然后 `make run` 一下，可以看到磁盘信息正确读取了！

```
[INFO] ysos_kernel::drivers::ata - Initialized ATA Buses.
[INFO] ysos_kernel::drivers::ata::bus - Identifying drive 0
[INFO] ysos_kernel::drivers::ata - Drive QEMU HARDDISK QM00001 (504 MiB) opened
[+] App list:
  App Name        | Size       | Entry Point
  ----------------+------------+--------------------
  counter         |    38992 B | 0x0000111100001470
  mq              |    38904 B | 0x00001111000017e0
  fork_test       |    35456 B | 0x00001111000011d0
  sleep           |    34416 B | 0x00001111000010a0
  hello           |    34320 B | 0x0000111100001020
  dinner          |    34480 B | 0x00001111000014f0
  fish            |    38792 B | 0x0000111100001320
  factorial       |    34872 B | 0x0000111100001490
  sh              |    35160 B | 0x0000111100001530

Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>]
```

2.3. 读写数据

我们要实现的是 `read_pio` 和 `write_pio` 函数，这里参考 `FIXME` 中的提示进行补全代码即可。这里使用了 `.to_le_bytes()` 函数来把一个 `word` 按照小端序拆成两个字节，以及使用 `from_le_bytes()` 把两个字节按照小端序拼成一个 `word`。`FIXME` 部分代码如下

```rust
// FIXED: read the data from the data port into the buffer
//      - use `buf.chunks_mut(2)`
//      - use `self.read_data()`
//      - ! pay attention to data endianness
for chunk in buf.chunks_mut(2) {
    // use .to_le_bytes() to convert data endianness
    let data = self.read_data().to_le_bytes();
    chunk[0] = data[0];
    if chunk.len() > 1 {
        chunk[1] = data[1];
    }
}


// FIXED: write the data from the buffer into the data port
//      - use `buf.chunks(2)`
//      - use `self.write_data()`
//      - ! pay attention to data endianness
for chunk in buf.chunks(2) {
    let mut data = [0u8; 2];
    data[0] = chunk[0];
    if chunk.len() > 1 {
        data[1] = chunk[1];
    }
    self.write_data(u16::from_le_bytes(data));
}
```

然后要用上面这些实现的命令来实现块设备，`block_count` 直接返回 `blocks` 的值即可；`read_block` 和 `write_block` 需要先获取总线的锁，然后调用对应的 `read_pio()` 或者 `write_pio()` 即可。注意这里 `read_pio` 需要传入一个可变的引用，因为要把值从端口传入缓冲区。这部分代码请见 `ata/mod.rs`。

最后需要实现外层的块设备，这里的 `read_block` 和 `write_block` 只需要根据分区的起始 `LBA` 以及块内偏移计算出物理块地址，然后委托给内层的物理块设备即可，代码请见 `partition/mod.rs`。

为了可以输出首个分区的类型信息，需要在 `partition` 中的 `Debug` trait 中的 `fmt` 加入一个 `type` 作为输出，这个需要在 `partition` 的定义中多增加一个成员 `part_type`，并且在 `new` 的时候加入把这个 `type_partition` 也传入进去。

然后我们把日志等级设置为 `Trace`，`make run` 一下可以看到，我们可以正确的在 `partition` 中得到首个分区的相关信息。

```
[TRACE] ysos_kernel::drivers::ata - Opening drive 0@0...
[INFO] ysos_kernel::drivers::ata - Initialized ATA Buses.
[INFO] ysos_kernel::drivers::ata::bus - Identifying drive 0
[INFO] ysos_kernel::drivers::ata - Drive QEMU HARDDISK QM00001 (504 MiB) opened
[TRACE] ysos_storage::partition::mbr - Partition 0: Partition Meta Data {
    Active: true,
    Begin Head: "0x01",
    Begin Sector: "0x0001",
    Begin Cylinder: "0x0000",
    Partition Type: "0x06",
    End Head: "0x0f",
    End Sector: "0x003f",
    End Cylinder: "0x03ff",
    Begin LBA: "0x0000003f",
    Total LBA: "0x000fbfc1",
}
[INFO] ysos_kernel - Partition 0 Info:Partition {
    type: "0x06",
    offset: 63,
    size: 1032129,
}
```

可以看到，`parse` 得到的第 0 个分区的偏移量与 Begin LBA 对应，大小 size 和类型 type 也和 `MBR` 中的信息一致。这里的 `0x06` 类型指的是 FAT16B 类型，是 FAT16 文件格式的一种。

3. FAT16 文件系统

3.1. BPB

这部分我们要做的是根据文档中对 BPB(Bios Parameter Block) 的结构定义来实现 `Fat16Bpb` 的定义，这部分代码请见 `bpb.rs`，这里不再赘述。实现完之后注释掉 `mod.rs` 中对其他模块的引用之后，运行测试，可以看到测试均可以通过！

```
(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage/src/fs/fat16$ cargo test --pack
age ysos_storage bpb::tests
ore: `_buf`

warning: `ysos_storage` (lib test) generated 4 warnings (run `cargo fix --lib -p ysos_st
orage --tests` to apply 4 suggestions)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.19s
     Running unittests src/lib.rs (/home/linkzelda/OS-lab/target/debug/deps/ysos_storage
-a8f099cc3289439a)

running 2 tests
test fs::fat16::bpb::tests::test_fat16_bpb_1 ... ok
test fs::fat16::bpb::tests::test_fat16_bpb_2 ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out; finished in
0.00s
```

3.2. DirEntry

首先我们需要实现 `is_directory` 和 `is_long_name` 函数，这个使用 `bitflag!` 宏配合 `.contains()` 判断文件属性的对应位即可。还需要实现一个 `is_valid` 函数，一个目录项合法需要它同时满足不是文件目录末尾，且该文件未被删除，用 `is_eod` 和 `is_unused` 判断即可。

然后要实现对目录项的解析，按照文档中的结构定义实现即可，这里需要对时间解析为 `FsTime` 类型，因为这里只需要精确到秒，因此不需要解析毫秒位。将 `date` 放在高 16 位，`time` 放在低 `16` 位，然后在 `parse_datatime` 中进行对应的解析即可（这里年份是从 `1980` 年开始的偏移，因此需要加上 `1980`）

然后需要实现对短文件名的解析，这里根据代码中的注释进行实现即可，主要就是将文件名 `basename` 和扩展名 `ext` 分开，然后检查长度/是否存在非法字符/格式是否正确等，如果出现非法则返回异常，否则返回解析出来的文件名和扩展名。

上述的代码请见 `direntry.rs`，这里不再赘述。实现完之后按照之前类似的方法，进行测试，可以看到通过了对目录项解析的测试！

```
warning: `ysos_storage` (lib test) generated 4 warnings (run `cargo fix --lib -p ysos_stor
age --tests` to apply 4 suggestions)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.59s
     Running unittests src/lib.rs (/home/linkzelda/OS-lab/target/debug/deps/ysos_storage-a
8f099cc3289439a) …

running 1 test
test fs::fat16::direntry::tests::test_dir_entry ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 3 filtered out; finished in 0.
00s

(base) linkzelda@LAPTOP-QFAEHS2I:~/OS-lab/crates/storage/src/fs/fat16$
```

3.3. Fat16Impl

首先我们要算出根目录占用的扇区个数以及起始扇区号，前者可以直接根据目录项的数目乘以每个目录占用的字节数 32 然后除以扇区大小即可（注意这里需要上取整），后者可以根据保留区大小以及 FAT 的大小计算出来。

然后要实现簇号到扇区号的转换，由于 FAT 的前两个条目是保留的，因此对于簇号为 `c` 的簇的真实簇号为 `c-2`，这里还需要判断簇号的是否在合法的范围内，然后计算出该簇对应的首个扇区。

此外，我们还实现了若干辅助函数，`next_cluster` 用于提取当前簇的下一个簇，方法是先计算出当前簇在 `FAT` 表中的偏移以及当前 `FAT` 表所在的扇区，然后提取这个位置的值就是下一簇号了（还需要判断一下是否为损坏的簇或者是已到达 `FAT` 表的末尾）

还有 `get_dir_entries` 函数，用于读取给定目录下的所有条目的信息，对于根目录，存储在 `FAT` 区后数据区之前，我们可以直接把这部分所有的扇区读完；对于子目录，可能会横跨多个簇，因此需要把该簇里面所有的扇区读完之后，再通过之前实现的 `next_cluster` 函数去到下一个簇。

最后还实现了一个 `find_entry` 函数，用于查找某个路径下的目录项，我们将路径拆分成文件/文件夹名路径，然后每次再当前目录下用 `get_dir_entries` 取出所有条目来寻找是否存在路径指定的下一个目录，如果存在则跳转到这个目录之后继续寻找。

接下来，需要实现 `Fat16` 的文件系统，对于 `read_dir`，我们直接调用 `find_entry` 函数即可，得到路径对应的目录后再使用 `get_dir_entries` 获取所有的条目并返回其 `Vec` 的迭代器；对于 `open_file`，也是使用 `find_entry` 得到目录，然后返回该目录的 `FileHandle`；对于 `metadata`，我们直接调用 `find_entry` 和 `.as_meta` 获得 `metadata` 即可，根目录是没有 `metadata` 的，因此我们要手动构造一个；最后的 `exists` 只要用 `find_entry` 看看是否可以取得一个合法目录即可。

上述代码请见 `fs/fat16/impls.rs`。

最后，要为 `File` 实现 `Read` trait，即把一个文件的内容读取到缓冲区中，具体而言，就是从当前文件指针指向的地址开始向后读取，知道缓冲区满或者抵达 `EOF` 为止。我们计算出当前地址偏移对应的簇偏移，扇区偏移以及扇区内偏移然后读取之后复制到缓冲区即可，如果在多个簇内，还需要使用 `next_cluster` 去到下一个簇中，中间需要注意复制的文件字节数不要超过缓冲区大小。

这部分代码请见 `fs/fat16/file.rs`。

4. 接入操作系统

这部分我们需要实现 `ls` 和 `cat` 命令。

4.1. 列出目录

首先我们实现 `ls` 命令，我们要做的就是把当前路径下获取的目录条目信息打印出来，具体的格式详见 `drivers/filesystem.rs`。

然后需要添加对应的系统调用给用户态使用，然后分发给 `sys_list_dir` 函数把路径转化为字符串，然后委托给我们上面实现的 `ls` 函数中即可，具体代码请见 `service.rs` 等。

4.2. 读取文件

我们采用 `Open - Read - Close` 过程，在 `sh` 中就是每次打开一个文件，然后一直用缓冲区进行读取，最后关闭该文件。

`sys_read` 我们先前已经实现了，因此我们只需要在之前的 `resource.rs` 中把新实现的 `FileHandle` 注册到里面即可，把 `read` 操作委托给文件句柄的 `read`。

因此我们只需要实现 `sys_open` 和 `sys_close`， 我们可以调用 `.open_file` 函数得到 `FileHandle`，然后插入到内核的 `ResourceSet` 中即可，关闭文件也是类似的，从 `ResourcesSet` 中删除该句柄即可。由于这里的 `ResourceSet` 是 `pub(super)` 的，因此我们在 `ProcessData` 中加入了 `open` 和 `close` 函数封装了对外的对应操作。

最后我们在 `lib.rs` 中加入文件系统的初始化，并在 `sh` 中加上 `ls` 和 `cat` 指令的操作，调用对应的系统调用即可。

我们可以看到可以通过 `ls` 和 `cat` 指令正确读取出 `eps/hello.txt` 的内容！

